

To complete your full CRUD API, you will add the update and delete logic. These endpoints manipulate your in-memory list based on the path ID. Below is the final code for both lanes.

Node.js Lane (Express)

Update your `server.js` file:

javascript

```
const express = require('express');
const app = express();
const PORT = 3000;

app.use(express.json());

let tasks = [
  { id: 1, title: "Buy groceries", done: false },
  { id: 2, title: "Clean the room", done: true },
  { id: 3, title: "Study API design", done: false }
];

app.get('/', (req, res) => {
  res.json({ name: "Task API", version: "1.0", endpoints: ["/tasks"] });
});

app.get('/health', (req, res) => {
  res.json({ status: "ok" });
});

app.get('/tasks', (req, res) => {
  res.json(tasks);
});

app.get('/tasks/:id', (req, res) => {
  const taskId = parseInt(req.params.id);
  const task = tasks.find(t => t.id === taskId);
  if (!task) {
    return res.status(404).json({ error: `Task ${taskId} not found` });
  }
  res.json(task);
});

app.post('/tasks', (req, res) => {
```

```
const { title } = req.body;
if (!title || title.trim() === "") {
  return res.status(400).json({ error: "Title is required" });
}
const nextId = tasks.length > 0 ? Math.max(...tasks.map(t => t.id)) + 1 : 1;
const newTask = { id: nextId, title: title, done: false };
tasks.push(newTask);
res.status(201).json(newTask);
});
```

// PUT /tasks/:id - Update an existing task

```
app.put('/tasks/:id', (req, res) => {
  const taskId = parseInt(req.params.id);
  const taskIndex = tasks.findIndex(t => t.id === taskId);

  if (taskIndex === -1) {
    return res.status(404).json({ error: `Task ${taskId} not found` });
  }
}
```

```
const { title, done } = req.body;
```

// Validate body is not totally empty

```
if (title === undefined && done === undefined) {
  return res.status(400).json({ error: "Missing fields to update" });
}
```

// Validate title if it is being updated

```
if (title !== undefined && (!title || title.trim() === "")) {
  return res.status(400).json({ error: "Title cannot be empty" });
}
```

// Validate done if it is being updated

```
if (done !== undefined && typeof done !== "boolean") {
  return res.status(400).json({ error: "Done must be a boolean" });
}
```

// Update properties if provided

```
if (title !== undefined) tasks[taskIndex].title = title;
if (done !== undefined) tasks[taskIndex].done = done;
```

```
res.json(tasks[taskIndex]);
});
```

// DELETE /tasks/:id - Delete a task

```

app.delete('/tasks/:id', (req, res) => {
  const taskId = parseInt(req.params.id);
  const taskIndex = tasks.findIndex(t => t.id === taskId);

  if (taskIndex === -1) {
    return res.status(404).json({ error: `Task ${taskId} not found` });
  }

  tasks.splice(taskIndex, 1);
  res.status(204).send(); // 204 No Content has an empty body
});

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});

```

Use code with caution.

● Python Lane (FastAPI)

Update your `main.py` file:

```

python

from fastapi import FastAPI, HTTPException, status, Response
from pydantic import BaseModel, Field
from typing import Optional

app = FastAPI()

# Pydantic model for creation
class TaskCreate(BaseModel):
    title: str = Field(..., min_length=1)

# Pydantic model for partial updates
class TaskUpdate(BaseModel):
    title: Optional[str] = Field(None, min_length=1)
    done: Optional[bool] = None

tasks = [
    {"id": 1, "title": "Buy groceries", "done": False},

```

```
{ "id": 2, "title": "Clean the room", "done": True },
{ "id": 3, "title": "Study API design", "done": False }
]

@app.get("/")
def read_root():
    return { "name": "Task API", "version": "1.0", "endpoints": [ "/tasks" ] }

@app.get('/health')
def read_health():
    return { "status": "ok" }

@app.get("/tasks")
def read_tasks():
    return tasks

@app.get("/tasks/{id}")
def read_task(id: int):
    task = next((t for t in tasks if t["id"] == id), None)
    if task is None:
        raise HTTPException(status_code=404, detail={"error": f"Task {id} not found"})
    return task

@app.post("/tasks", status_code=status.HTTP_201_CREATED)
def create_task(task_input: TaskCreate):
    if not task_input.title.strip():
        raise HTTPException(status_code=400, detail={"error": "Title cannot be empty space"})
    next_id = max([t["id"] for t in tasks], default=0) + 1
    new_task = { "id": next_id, "title": task_input.title, "done": False }
    tasks.append(new_task)
    return new_task

# PUT /tasks/{id} - Update an existing task
@app.put("/tasks/{id}")
def update_task(id: int, task_input: TaskUpdate):
    task = next((t for t in tasks if t["id"] == id), None)
    if task is None:
        raise HTTPException(status_code=404, detail={"error": f"Task {id} not found"})

    # Check if empty body was sent
    if task_input.title is None and task_input.done is None:
        raise HTTPException(status_code=400, detail={"error": "Missing fields to update"})

    if task_input.title is not None:
```

```

    if not task_input.title.strip():
        raise HTTPException(status_code=400, detail={"error": "Title cannot be empty s
    task["title"] = task_input.title

    if task_input.done is not None:
        task["done"] = task_input.done

    return task

# DELETE /tasks/{id} - Delete a task
@app.delete("/tasks/{id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_task(id: int):
    global tasks
    task = next((t for t in tasks if t["id"] == id), None)
    if task is None:
        raise HTTPException(status_code=404, detail={"error": f"Task {id} not found"})

    tasks = [t for t in tasks if t["id"] != id]
    return Response(status_code=status.HTTP_204_NO_CONTENT)

```

Use code with caution.

🔍 Checkpoint Verification

Run this curl sequence to test your complete CRUD system (adjust port to `8000` for Python):

- Create:** `curl -i -X POST http://localhost:3000/tasks -H "Content-Type: application/json" -d '{"title":"Test Task"}'` (Expect `201`)
- Update:** `curl -i -X PUT http://localhost:3000/tasks/4 -H "Content-Type: application/json" -d '{"title":"Updated Title", "done":true}'` (Expect `200`)
- Delete:** `curl -i -X DELETE http://localhost:3000/tasks/4` (Expect `204` with no body text)
- Verify Missing:** `curl -i http://localhost:3000/tasks/4` (Expect `404`)

💾 Git Commit

Save your completed CRUD setup:

```
bash
```

```
git add .  
git commit -m "Stage 4: full CRUD"
```

Use code with caution.

Congratulations on building your very first complete backend engine! If you want to keep expanding this application, would you like to explore **adding query filters**